

ОТЧЕТ

прохождения производственной практики (по профилю специальности)
(вид (тип) практики) (наименование практики)

обучающегося очной формы обучения 3 курса группы K0709-23/1

по программе среднего профессионального образования – программе
подготовки специалистов среднего звена по специальности

09.02.07 Информационные системы и программирование
(направление подготовки, специальность)

_____ Галактионовой Екатерины Александровны _____
(фамилия, имя и отчество обучающегося)

Наименование организации и ее местонахождение:
ООО «Газпром ЦПС», г. Санкт-Петербург, пр-кт Добролюбова, д. 16, к.

Сроки прохождения практики с «25» мая 2026 г. по «21» июня 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. ХАРАКТЕРИСТИКА МЕСТА ПРОХОЖДЕНИЯ ПРАКТИКИ	3
2. ВЫПОЛНЕНИЕ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ	3
2.1. Задание 1. Фильтр Docker-образов перед репликацией в prod	3
2.2. Задание 2. Синхронизация инвентаря zVirt → NetBox	4
2.3. Задание 3. Автоматизация данных из корпоративного каталога	5
ЗАКЛЮЧЕНИЕ	7
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	7

ВВЕДЕНИЕ

Целью учебной практики являлось закрепление теоретических знаний и формирование профессиональных компетенций в области разработки программного обеспечения, DevOps и администрирования инфраструктуры. В ходе практики необходимо было выполнить два инженерных проекта на языке Python: лёгкий фильтр Docker-образов перед репликацией в production и двухконтурный конвейер синхронизации инвентаря виртуальных машин из zVirt в NetBox.

Задачи практики:

- изучить технические задания и существующие аналоги решений;
- спроектировать и реализовать утилиты согласно требованиям ТЗ;
- провести тестирование на эталонных сценариях;
- подготовить документацию и отчётные материалы.

1. ХАРАКТЕРИСТИКА МЕСТА ПРОХОЖДЕНИЯ ПРАКТИКИ

Практика проходила в ООО «Газпром ЦПС», г. Санкт-Петербург, пр-кт Добролюбова, д. 16, к. 2

. В качестве базы для выполнения заданий использовался учебно-производственный проект, моделирующий задачи инфраструктурной безопасности и учёта ИТ-ресурсов. Рабочее место включало персональный компьютер с установленными средами разработки.

2. ВЫПОЛНЕНИЕ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

2.1. Задание 1. Фильтр Docker-образов перед репликацией в prod

Контекст задачи: в инфраструктуре существует тяжёлый сканер контейнеров, который при росте нагрузки перегружается. Требовалось разработать лёгкий предварительный фильтр, проверяющий образ до push в production registry и отсекающий секреты, сертификаты, РНР-файлы, логи и тестовые артефакты.

Реализован проект `filtr-docker` (каталог `1/filtr-docker`) со следующей структурой:

- `scan-image.sh` — `bash`-обёртка для CI с созданием `workdir` и `cleanup`;
- `scanner.py` — экспорт образа через `docker save`, распаковка слоёв `OCI/tar`, обход ФС;
- `rules.py` — правила `CRITICAL/WARNING`, фильтр слоя приложения;
- `report.py` — текстовый и `JSON`-отчёт, коды выхода `0/1/2`;
- `guidance.py` — рекомендации при `FAILED`;
- `dashboard_server.py` и `webui` — веб-интерфейс `Control Tower`;
- `REST API` — `/api/health`, `/api/scan`, `/api/scans`.

Для проверки подготовлены эталонные образы `bad-container` (нарушения: `.env`, `cert.pem`, `shell.php`, `debug.log`, `test.sh`, `backup.bak`) и `clean-container` (минимальный `runtime`). Результаты: `bad-container:test` → `FAILED` (3 critical, 3 warning); `clean-container:test` → `PASSED`. Проверка выполнена через `CLI`, веб-UI и `REST API` (`Kreya`). Проверка также осуществлялась на рабочем контейнере, с известными и распространёнными проблемами. Проверки пройдены успешно.

2.2. Задание 2. Синхронизация инвентаря `zVirt` → `NetBox`

Контекст задачи: `zVirt` находится во внешнем контуре, `NetBox` — во внутреннем. Прямой доступ к `API zVirt` из внутренней сети запрещён. Требовалось построить конвейер: `export-tool` (внешний контур) → `JSON v2.0` → `merge-tool` (внутренний контур) → `NetBox API`.

Реализованы два приложения:

`export-tool` (каталог `2/export-tool`):

- подключение к `zVirt` через `ovirtsdk4`;

- сбор метаданных VM: vcpus, memory, cluster, status, disks;
- формирование JSON v2.0 с MD5-checksum;
- CLI с флагами --output-dir, --verbose, --no-ip.

merge-tool (каталог 2/merge-tool):

- загрузка JSON-снимков в Postgres;
- трёхстороннее сравнение base → new → NetBox;
- типы изменений: new, deleted, updated, conflict;
- маппинг статусов zVirt ↔ NetBox;
- применение изменений через pynetbox;
- история мержей и откат (rollback);
- веб-интерфейс на Streamlit.

Проведено тестирование на sample_base.json и sample_new.json: сценарии чистого мержа, конфликта полей и отката к состоянию NetBox до применения изменений.

По итогам тестирования конвейер zVirt → NetBox работает стабильно: экспорт снимка, трёхстороннее сравнение, разрешение конфликтов, мерж и откат выполняются в соответствии с ТЗ. Вместе с тем для промышленной эксплуатации целесообразно доработать синхронизацию IP-интерфейсов, режим dry-run, автоматизацию переноса JSON между контурами, планировщик регулярных запусков и экспорт отчётов сравнения для аудита.

2.3. Задание 3. Автоматизация данных из корпоративного каталога

Контекст задачи: разработать скрипт автоматизированной выгрузки сведений о пользователях и группах доступа из корпоративного каталога Active Directory по протоколу LDAP.

Администраторам нужен регулярный срез учётных записей и членства в группах безопасности — для аудита и контроля доступа к корпоративным системам. Ручной сбор через оснастку AD или PowerShell неудобен при нескольких доменах и большом числе групп.

Реализация:

Разработан скрипт `ad_users.py` на Python с библиотекой `python-ldap`.

Подключение к AD:

- протокол LDAP v3, шифрование LDAPS (порт 636);
- аутентификация службы: `bind_dn` в `config.json`, пароль — в `.ldap_pass`;
- обход нескольких доменов из списка `ldap_servers`.

Поиск данных:

- группы — по маскам ECP, EZ, GE, GR;
- пользователи — члены найденных групп (обработка пакетами по 150 групп);
- пагинация через `SimplePagedResultsControl` (1000 записей на страницу).

Собираемые атрибуты:

- пользователи: `login`, статус УЗ, даты последнего входа;
- группы: наименование, описание, расположение в AD, система, контур (prod/test).

Обработка: определение статуса УЗ по `userAccountControl`, маппинг OU на наименование системы, определение контура по имени группы.

Итог: реализован автоматизированный сбор пользователей и групп из нескольких доменов AD.

ЗАКЛЮЧЕНИЕ

В ходе практики выполнены все пункты индивидуального задания. Разработаны три работоспособных программных комплекса, решающих задачи инфраструктурной безопасности и актуализации CMDB. Получены практические навыки работы с Docker, REST API, Postgres, Streamlit, интеграцией с NetBox и zVirt, а также протоколом LDAP v3, шифрованием LDAPS.

В процессе практики сформированы компетенции:

- анализ технического задания и выбор архитектурного решения;
- разработка CLI-утилит и веб-интерфейсов на Python;
- тестирование и документирование программного обеспечения;
- работа осуществлялась в закрытых контурах со всеми ограничениями.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Техническое задание «Фильтр Docker-образов перед репликацией в production».
2. Техническое задание «Синхронизация инвентаря zVirt → NetBox».
3. Документация Docker: <https://docs.docker.com/>
4. Документация NetBox: <https://docs.netbox.dev/>
5. Документация oVirt/zVirt API
6. Документация Streamlit: <https://docs.streamlit.io/>
7. Документация Python 3.11: <https://docs.python.org/3/>